

Project Overview

Over de komende vijf middagen bouw je je eigen **REST API**. Je begint met de eerste resources en endpoints, en bouwt daar stap voor stap op verder: volledige CRUD, nette foutafhandeling, API design principes, authenticatie en autorisatie. En uiteraard, op de laatste dag presenteer je het eindresultaat aan een groter publiek.

Je kiest zelf een domein voor je API. Denk aan iets wat je zou willen gebruiken. Het kan van alles zijn: een Pokémon encyclopedie, een kaaswinkel, een filmarchief, een receptenboek, of iets heel anders. Dat domein gebruik je de rest van het blok voor het middagproject, dus kies iets wat genoeg variatie biedt om meerdere resources en relaties te ontdekken en wat je niet beu bent na middag 2.

Het startproject gebruikt Jakarta REST en draait op een embedded server. Persistence doe je met een H2 inmemory database, net als in eerdere modules. Testen kun je kiezen voor Postman of Bruno (naast unit tests).

Middag 1: Domein, resources en de eerste endpoints



Doel

We gaan beginnen met nadenken over je domein en je resources, voordat je ook maar één regel code schrijft. Je maakt keuzes die de rest van het project beïnvloeden: welke dingen bestaan er in jouw systeem, hoe noem je ze, en hoe geef je ze een plek in een URL?

Aan het eind van de middag heb je een draaiende API met je eerste echte endpoints, en heb je die al live getest vanuit Postman of Bruno.

Leerdoelen

- Een domeinmodel definiëren en vertalen naar REST-resources met betekenisvolle URL's
- Werkende GET-endpoints implementeren met Jakarta REST (`@GET`, `@Path`, `@Produces`)
- Een HTTP-response inspecteren in Postman: status, headers en body analyseren
- Beredeneren waarom een resource-georiënteerde aanpak verschilt van een actie-georiënteerde aanpak

Projectonderdelen

De middag bevat **3 onderdelen**.

Onderdeel 1: Domein bepalen en eerste endpoint live zetten

Begin met het kiezen van een domein als groep. Schrijf op welke resources je hebt: dat zijn de "dingen" in jouw systeem (zelfstandige naamwoorden, geen werkwoorden). Denk na over welke twee of drie resources je de komende weken gaat bouwen, en welke relaties daartussen bestaan.

Kies daarna één resource om vandaag mee te beginnen. Maak een Java record voor die resource en voeg de eerste twee endpoints toe:

- `GET /<jouw-resource>` (geeft alle items terug)
- `GET /<jouw-resource>/{id}` (geeft één item terug obv ID)

Gebruik het api-starter project als vertrekpunt. Verwijder de voorbeeldcode als je je eigen resource hebt.

Succescriteria

- Het domeinmodel is opgeschreven: minimaal twee resources met een beschrijving
- `GET /<resource>` geeft een JSON-array terug met minimaal drie items
- `GET /<resource>/{id}` geeft het juiste item terug bij een bestaand ID
- Beide endpoints zijn werkend getest in Postman of Bruno
- De response bevat een `Content-Type: application/json` header

Onderdeel 2: Tweede resource toevoegen

Voeg een tweede resource toe aan je API. Deze resource hoeft nog geen relatie te hebben met de eerste (dat komt later wel). Zorg wel dat de URL-structuur consistent is met de eerste resource.

Test ook wat er nu terugkomt als je een ID opvraagt dat niet bestaat. Wat is het gedrag? Zou je dit willen veranderen? (Hoef je nu nog niet zo te implementeren.)

Succescriteria:

- De tweede resource heeft ook een `GET all` en `GET by ID` endpoint
- Beide endpoints zijn getest en werken
- Je hebt genoteerd wat er terugkomt bij een niet-bestaand ID, en kunt uitleggen of dat gewenst gedrag is

Onderdeel 3: Postman-collectie aanmaken

Maak een Postman- of Bruno-collectie aan voor je API. Voeg alle requests die je vandaag hebt gebouwd toe als aparte requests in de collectie.

Voeg ook minstens één request toe dat bewust fout gaat: een ID dat niet bestaat, een verkeerd pad, of een methode die niet ondersteund wordt. Inspecteer de response aandachtig: welke statuscode krijg je? Wat staat er in de body?

Succescriteria

- Collectie bevat alle werkende requests van vandaag
- Minstens één "fout" request is toegevoegd en gedocumenteerd

Elk request heeft een duidelijke naam zodat anderen begrijpen wat het doet

Bonuschallenge (Optioneel)

Voeg een Postman of Bruno environment toe voor je lokale omgeving, zodat de base URL (<http://localhost:8080>) een variabele is in plaats van een hardcoded waarde in elk request. Bedenk waarom dit nuttig is zodra je de API ook ergens anders zou willen deployen.

